C++ Mastery Guide

From Tutorial Completer to Software Builder

Learn to Think, Design, and Code Real Programs

Progressive Learning Path

For Students Who Know Basics But Fear Real Projects : Kundan Maurya

February 23, 2026

Contents

Preface **5**

I The Art of Breaking Down Problems **7**

1 How Professional Programmers Think **9**
1.1 The 4-Step Problem Solving Method 9

2 Your First Real Program: Student Database **11**
2.1 Version 1: The Bare Minimum (Understand This Completely) 11
2.2 Version 2: Multiple Students (Arrays) 12
2.3 Version 3: Adding Functions (Procedural Approach) 13
2.4 Version 4: OOP - Student as a Class 15
2.5 Version 5: Manager Class - Separation of Concerns 16

3 Project 1: Complete Library Management System **21**
3.1 Requirements Analysis (Thinking Process) 21
3.2 Step-by-Step Implementation 21
3.2.1 Step 1: Book Class (Test Independently) 21
3.2.2 Step 2: Member Class 23
3.2.3 Step 3: Library Manager (The Big Class) 25
3.2.4 Step 4: Complete Main Program 27

4 Project 2: Banking System - Inheritance and Polymorphism **31**
4.1 Learning Focus 31
4.2 Step-by-Step Build 31
4.2.1 Step 1: Abstract Account Class 31
4.2.2 Step 2: Derived Account Types 33
4.2.3 Step 3: Bank Class (Polymorphic Container) 34
4.2.4 Step 4: Main Program Showing Polymorphism 36

5 Project 3: E-Commerce Console Application **39**
5.1 The Ultimate Integration Project 39
5.2 Core Classes Overview 39
5.3 The Complete System 43
5.4 Sample Usage (Main Program) 44

6 How to Continue Learning **49**
6.1 Self-Learning Strategies 49

How to Use This Guide

If you've completed C++ tutorials and STL basics but freeze when asked to "write a real program," this guide is for you.

The Problem: Tutorials teach syntax in isolation. Real software needs you to combine 20 concepts while making design decisions.

Our Solution: We evolve programs gradually. Each chapter starts with a 20-line version you understand, then adds features one by one, explaining the thinking at each step.

Pro Tip: Mindset Shift

The Golden Rule: Never copy-paste. Type every line yourself. When you get an error, that's your real learning moment.

Part I

The Art of Breaking Down Problems

Chapter 1

How Professional Programmers Think

1.1 The 4-Step Problem Solving Method

How to Think About This Problem

Step 1: Understand the Problem Deeply
Before writing any code, write the problem in plain English. If you can't explain it to a friend, you don't understand it.
Example - Student Management System:

- What is a "student"? (Data: name, roll, marks)
- What can we do with students? (Add, remove, search, display)
- Who uses this? (Admin - single user for now)
- Data persistence? (No, memory only for first version)

Step 2: Identify Nouns and Verbs
Nouns become **classes**. Verbs become **functions**.

Nouns (Classes)	Verbs (Methods)
Student	addStudent(), display(), calculateGrade()
Database/Manager	add(), remove(), find(), listAll()

How to Think About This Problem

Step 3: Start Small - The MVP Approach
MVP = Minimum Viable Product. The smallest version that works.
Wrong approach: Plan every feature for 2 weeks, then code.
Right approach:

1. Version 1: Store one student, display it (20 lines)
2. Version 2: Store 10 students in array (40 lines)
3. Version 3: Use vector, add search (60 lines)
4. Version 4: Add file storage (100 lines)

Each version gives you **working code** + **confidence**.

How to Think About This Problem

Step 4: Refactor Before Adding
When code becomes messy, clean it up. Don't pile features on garbage.
Signs you need refactoring:

- Function is longer than 30 lines
- Same code copied 3+ times
- Variable names don't make sense anymore
- You're afraid to touch a section

Chapter 2

Your First Real Program: Student Database

2.1 Version 1: The Bare Minimum (Understand This Completely)

Code Evolution: From Basic to Advanced

Goal: Store ONE student's data and print it.
Thinking Process:

- I need to group related data (name, roll, marks) → `struct`
- No functions needed yet, just main()
- Hardcode the data, no user input (simpler)

```cpp
// Version 1: Single student, hardcoded
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int rollNumber;
    float marks;
};

int main() {
    // Create one student
    Student s1;
    s1.name = "Rahul Kumar";
    s1.rollNumber = 101;
    s1.marks = 85.5;

    // Display
    cout << "Student Details:" << endl;
    cout << "Name: " << s1.name << endl;
    cout << "Roll: " << s1.rollNumber << endl;
    cout << "Marks: " << s1.marks << endl;
```

```
    return 0;
}
```

When You Get Stuck: Debug Strategy

Compile and run this. Understand every line.
Common errors at this stage:

- Forgot `#include <string>`
- Used `string` without `std::` or `using namespace std`
- Semicolon missing after struct

When stuck: Check C++ reference at cppreference.com → Search "struct"

2.2 Version 2: Multiple Students (Arrays)

Code Evolution: From Basic to Advanced

What changed? Need to store 3 students instead of 1.
Thinking:

- Array: `Student students[3]` - fixed size
- Need loop to input all
- Need loop to display all

```
// Version 2: Multiple students with arrays
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int rollNumber;
    float marks;
};

int main() {
    const int MAX = 3;
    Student students[MAX];

    // Input
    for(int i = 0; i < MAX; i++) {
        cout << "\nEnter details for student " << (i+1) << ":" << endl;
        cout << "Name: ";
        cin >> students[i].name;  // Problem: Can't enter "Rahul Kumar" (space issue)
        cout << "Roll: ";
        cin >> students[i].rollNumber;
        cout << "Marks: ";
```

```
        cin >> students[i].marks;
    }

    // Display all
    cout << "\n=== All Students ===" << endl;
    for(int i = 0; i < MAX; i++) {
        cout << students[i].name << " | "
             << students[i].rollNumber << " | "
             << students[i].marks << endl;
    }

    return 0;
}
```

Common Beginner Mistake

The Space Problem: `cin » string` stops at space. "Rahul Kumar" becomes just "Rahul".
Solution: Use `getline(cin, str)` but need to handle buffer clearing.
Don't fix this yet. First understand the array version completely.

2.3 Version 3: Adding Functions (Procedural Approach)

Code Evolution: From Basic to Advanced

Problem with Version 2: All code in main(). Messy and not reusable.
Thinking:

- Input logic → `inputStudent()` function
- Display logic → `displayStudent()` function
- Maybe a function to calculate grade?

```
// Version 3: Functions for organization
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int rollNumber;
    float marks;
};

// Function declarations
void inputStudent(Student &s);       // Pass by reference to modify
void displayStudent(const Student &s); // Pass by const ref (read-only)
char calculateGrade(float marks);

int main() {
    const int MAX = 3;
    Student students[MAX];
```

```

    // Input
    for(int i = 0; i < MAX; i++) {
        cout << "\nStudent␣" << (i+1) << ":" << endl;
        inputStudent(students[i]);
    }

    // Display with grades
    cout << "\n===␣Results␣===" << endl;
    for(int i = 0; i < MAX; i++) {
        displayStudent(students[i]);
        char grade = calculateGrade(students[i].marks);
        cout << "Grade:␣" << grade << endl;
    }

    return 0;
}

// Function definitions
void inputStudent(Student &s) {
    cout << "Name:␣";
    cin >> s.name;
    cout << "Roll:␣";
    cin >> s.rollNumber;
    cout << "Marks:␣";
    cin >> s.marks;
}

void displayStudent(const Student &s) {
    cout << s.name << "␣(Roll:␣" << s.rollNumber << ")␣-␣"
         << s.marks << "␣marks" << endl;
}

char calculateGrade(float marks) {
    if(marks >= 90) return 'A';
    if(marks >= 80) return 'B';
    if(marks >= 70) return 'C';
    if(marks >= 60) return 'D';
    return 'F';
}
```

Pro Tip: Mindset Shift

Key Insight: Functions separate **what** from **how**.
Main says "input a student" not "cout name, cin name, cout roll, cin roll..."
This is the first step toward professional code.

2.4 Version 4: OOP - Student as a Class

Code Evolution: From Basic to Advanced

Why upgrade from struct to class?
Struct just holds data. Class holds data **and** the operations on that data.
Thinking:

- Student should know how to display itself
- Student should calculate its own grade
- Data should be private (controlled access)

```cpp
// Version 4: Student as a proper class
#include <iostream>
#include <string>
#include <vector>  // Now we use STL!
using namespace std;

class Student {
private:
    string name;
    int rollNumber;
    float marks;

public:
    // Constructor - creates object with data
    Student(string n, int roll, float m)
        : name(n), rollNumber(roll), marks(m) {
        // Member initializer list (: part) is preferred
    }

    // Getters (accessors)
    string getName() const { return name; }
    int getRoll() const { return rollNumber; }
    float getMarks() const { return marks; }

    // Setters (mutators) - with validation
    void setMarks(float m) {
        if(m >= 0 && m <= 100) {
            marks = m;
        } else {
            cout << "Invalid marks!" << endl;
        }
    }

    // Behavior methods
    char getGrade() const {
        if(marks >= 90) return 'A';
        if(marks >= 80) return 'B';
        if(marks >= 70) return 'C';
        if(marks >= 60) return 'D';
        return 'F';
    }

```

```
    void display() const {
        cout << name << " (" << rollNumber << "): "
             << marks << " [" << getGrade() << "]" << endl;
    }
};

int main() {
    vector<Student> students;  // Dynamic array from STL

    // Add students
    students.push_back(Student("Rahul", 101, 85.5));
    students.push_back(Student("Priya", 102, 92.0));
    students.push_back(Student("Amit", 103, 78.5));

    // Display all - using range-based for
    cout << "=== Student Report ===" << endl;
    for(const auto &s : students) {
        s.display();
    }

    // Find and modify
    students[0].setMarks(88.0);  // Update marks
    cout << "\nAfter update:" << endl;
    students[0].display();

    return 0;
}
```

When You Get Stuck: Debug Strategy

New concepts here:

- `vector` - Dynamic array that grows automatically
- `push_back()` - Add element
- `const` methods - Promise not to modify object
- Constructor with initializer list
- Range-based for loop: `for(const auto &s : students)`

When confused: Check cppreference.com → `vector`

2.5 Version 5: Manager Class - Separation of Concerns

Code Evolution: From Basic to Advanced

New Problem: Main is doing too much - creating students, storing them, displaying menu.
Professional Thinking:

- Student class: Knows about single student data

- StudentManager class: Knows about collection operations (add, find, delete)
- Main: Just coordinates, delegates to manager

This is **Single Responsibility Principle**.

```cpp
// Version 5: Manager class for collection operations
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>  // For find_if
using namespace std;

class Student {
    // Same as Version 4...
    string name;
    int rollNumber;
    float marks;

public:
    Student(string n, int roll, float m)
        : name(n), rollNumber(roll), marks(m) {}

    int getRoll() const { return rollNumber; }
    string getName() const { return name; }

    void display() const {
        cout << rollNumber << ": " << name << " (" << marks << ")" << endl;
    }
};

// NEW: Manager class handles collection
class StudentManager {
private:
    vector<Student> students;

public:
    void addStudent(const Student &s) {
        // Check for duplicate roll number
        auto it = find_if(students.begin(), students.end(),
            [&s](const Student &existing) {
                return existing.getRoll() == s.getRoll();
            });

        if(it != students.end()) {
            cout << "Error: Roll number " << s.getRoll() << " already exists!" << endl;
            return;
        }

        students.push_back(s);
        cout << "Student added successfully!" << endl;
    }

    void displayAll() const {
        if(students.empty()) {
```

```
            cout << "No students in database." << endl;
            return;
        }

        cout << "\n=== All Students ===" << endl;
        for(const auto &s : students) {
            s.display();
        }
    }

    void findStudent(int roll) const {
        auto it = find_if(students.begin(), students.end(),
            [roll](const Student &s) { return s.getRoll() == roll; });

        if(it != students.end()) {
            cout << "Found: ";
            it->display();
        } else {
            cout << "Student with roll " << roll << " not found." <<
                endl;
        }
    }

    void removeStudent(int roll) {
        auto it = remove_if(students.begin(), students.end(),
            [roll](const Student &s) { return s.getRoll() == roll; });

        if(it != students.end()) {
            students.erase(it, students.end());
            cout << "Student removed." << endl;
        } else {
            cout << "Student not found." << endl;
        }
    }

    size_t getCount() const { return students.size(); }
};

// Menu-driven interface
int main() {
    StudentManager manager;
    int choice;

    do {
        cout << "\n=== Student Database System ===" << endl;
        cout << "1. Add Student" << endl;
        cout << "2. Display All" << endl;
        cout << "3. Find Student" << endl;
        cout << "4. Remove Student" << endl;
        cout << "5. Exit" << endl;
        cout << "Choice: ";
        cin >> choice;

        switch(choice) {
            case 1: {
                string name;
                int roll;
                float marks;
```

```

                cout << "Enter name: ";
                cin >> name;  // Still has space issue, fix later
                cout << "Enter roll: ";
                cin >> roll;
                cout << "Enter marks: ";
                cin >> marks;

                manager.addStudent(Student(name, roll, marks));
                break;
            }
            case 2:
                manager.displayAll();
                break;
            case 3: {
                int roll;
                cout << "Enter roll to find: ";
                cin >> roll;
                manager.findStudent(roll);
                break;
            }
            case 4: {
                int roll;
                cout << "Enter roll to remove: ";
                cin >> roll;
                manager.removeStudent(roll);
                break;
            }
            case 5:
                cout << "Goodbye!" << endl;
                break;
            default:
                cout << "Invalid choice!" << endl;
        }
    } while(choice != 5);

    return 0;
}
```

Pro Tip: Mindset Shift

What we learned in this evolution:

1. **Version 1-2:** Data structures (struct, arrays)
2. **Version 3:** Functions for organization
3. **Version 4:** OOP basics (class, encapsulation, vector)
4. **Version 5:** Design patterns (Manager class, Single Responsibility)
5. **STL used:** vector, algorithm (find_if, remove_if), lambda

Total lines: 150, but covers 80% of OOP concepts!

Chapter 3

Project 1: Complete Library Management System

3.1 Requirements Analysis (Thinking Process)

How to Think About This Problem

Problem Statement: Build a library system where:

- Books can be added, removed, searched
- Members can borrow, return books
- Track who has which book
- Due date calculation

Nouns → Classes:

- Book (title, author, ISBN, isAvailable)
- Member (name, memberID, borrowedBooks)
- Transaction (who borrowed what, when, dueDate)
- Library (manages books and members)

Relationships:

- Library has many Books (composition)
- Library has many Members (composition)
- Member borrows many Books (association)

3.2 Step-by-Step Implementation

3.2.1 Step 1: Book Class (Test Independently)

```cpp
// book.hpp
#ifndef BOOK_HPP
#define BOOK_HPP

#include <string>
#include <iostream>

class Book {
private:
    std::string isbn;        // Unique identifier
    std::string title;
    std::string author;
    bool available;
    int publicationYear;

public:
    Book(std::string i, std::string t, std::string a, int year);

    // Getters
    std::string getISBN() const;
    std::string getTitle() const;
    std::string getAuthor() const;
    bool isAvailable() const;

    // Setters
    void setAvailable(bool status);

    // Display
    void display() const;

    // For searching
    bool matches(const std::string &keyword) const;
};

#endif
```

```cpp
// book.cpp
#include "book.hpp"

Book::Book(std::string i, std::string t, std::string a, int year)
    : isbn(i), title(t), author(a), publicationYear(year), available(
        true) {}

std::string Book::getISBN() const { return isbn; }
std::string Book::getTitle() const { return title; }
std::string Book::getAuthor() const { return author; }
bool Book::isAvailable() const { return available; }

void Book::setAvailable(bool status) { available = status; }

void Book::display() const {
    std::cout << "[" << isbn << "] " << title
              << " by " << author
              << " (" << (available ? "Available" : "Borrowed") << ")"
              << std::endl;
}

```

```cpp
bool Book::matches(const std::string &keyword) const {
    return title.find(keyword) != std::string::npos ||
           author.find(keyword) != std::string::npos ||
           isbn.find(keyword) != std::string::npos;
}
```

When You Get Stuck: Debug Strategy

Test this class independently before proceeding:

```cpp
// test_book.cpp
#include "book.hpp"

int main() {
    Book b1("978-3-16", "C++ Primer", "Stanley Lippman", 2012);
    Book b2("978-0-13", "Clean Code", "Robert Martin", 2008);

    b1.display();
    b2.display();

    // Test search
    if(b1.matches("C++")) {
        std::cout << "Found C++ book!" << std::endl;
    }

    // Test availability
    b1.setAvailable(false);
    b1.display();

    return 0;
}
```

Compile: `g++ -std=c++17 book.cpp test_book.cpp -o test_book`
Run and verify output before continuing.

3.2.2 Step 2: Member Class

```cpp
// member.hpp
#ifndef MEMBER_HPP
#define MEMBER_HPP

#include <string>
#include <vector>
#include <ctime>

class Book;  // Forward declaration

class Member {
private:
    int memberID;
    std::string name;
    std::string email;
    std::vector<std::string> borrowedISBNs;  // Just store ISBNs
    time_t registrationDate;

public:
```

```cpp
    Member(int id, std::string n, std::string e);

    int getID() const;
    std::string getName() const;

    void borrowBook(const std::string &isbn);
    bool returnBook(const std::string &isbn);
    bool hasBorrowed(const std::string &isbn) const;

    void display() const;
    size_t getBorrowedCount() const;
};

#endif
```

```cpp
// member.cpp
#include "member.hpp"
#include <iostream>

Member::Member(int id, std::string n, std::string e)
    : memberID(id), name(n), email(e), registrationDate(time(nullptr))
        {}

int Member::getID() const { return memberID; }
std::string Member::getName() const { return name; }

void Member::borrowBook(const std::string &isbn) {
    borrowedISBNs.push_back(isbn);
}

bool Member::returnBook(const std::string &isbn) {
    auto it = std::find(borrowedISBNs.begin(), borrowedISBNs.end(), isbn
        );
    if(it != borrowedISBNs.end()) {
        borrowedISBNs.erase(it);
        return true;
    }
    return false;
}

bool Member::hasBorrowed(const std::string &isbn) const {
    return std::find(borrowedISBNs.begin(), borrowedISBNs.end(), isbn)
           != borrowedISBNs.end();
}

void Member::display() const {
    std::cout << "Member␣#" << memberID << ":␣" << name
              << "␣(Books␣borrowed:␣" << borrowedISBNs.size() << ")"
              << std::endl;
}

size_t Member::getBorrowedCount() const {
    return borrowedISBNs.size();
}
```

3.2.3 Step 3: Library Manager (The Big Class)

```cpp
// library.hpp
#ifndef LIBRARY_HPP
#define LIBRARY_HPP

#include "book.hpp"
#include "member.hpp"
#include <vector>
#include <map>
#include <memory>

struct Transaction {
    int memberID;
    std::string isbn;
    time_t borrowDate;
    time_t dueDate;

    Transaction(int mid, std::string i, time_t borrow, int daysAllowed =
        14);
};

class Library {
private:
    std::vector<std::unique_ptr<Book>> books;       // Owned by library
    std::vector<std::unique_ptr<Member>> members;   // Owned by library
    std::vector<Transaction> transactions;

    // Quick lookup indexes
    std::map<std::string, Book*> isbnIndex;         // ISBN -> Book
        pointer

public:
    // Book management
    void addBook(std::string isbn, std::string title,
                 std::string author, int year);
    void removeBook(const std::string &isbn);
    Book* findBook(const std::string &isbn);
    void searchBooks(const std::string &keyword) const;
    void listAllBooks() const;

    // Member management
    void registerMember(int id, std::string name, std::string email);
    Member* findMember(int id);

    // Operations
    bool borrowBook(int memberID, const std::string &isbn);
    bool returnBook(int memberID, const std::string &isbn);

    // Reports
    void listOverdueBooks() const;
    void generateStatistics() const;
};

#endif
```

```cpp
// library.cpp (Key functions)
#include "library.hpp"
```

```cpp
#include <iostream>
#include <algorithm>

Transaction::Transaction(int mid, std::string i, time_t borrow, int days
    )
    : memberID(mid), isbn(i), borrowDate(borrow) {
    dueDate = borrow + (days * 24 * 60 * 60);  // days to seconds
}

void Library::addBook(std::string isbn, std::string title,
                      std::string author, int year) {
    if(isbnIndex.find(isbn) != isbnIndex.end()) {
        std::cout << "Book with ISBN " << isbn << " already exists!" <<
            std::endl;
        return;
    }

    auto book = std::make_unique<Book>(isbn, title, author, year);
    isbnIndex[isbn] = book.get();  // Store pointer for quick lookup
    books.push_back(std::move(book));

    std::cout << "Book added successfully!" << std::endl;
}

Book* Library::findBook(const std::string &isbn) {
    auto it = isbnIndex.find(isbn);
    if(it != isbnIndex.end()) {
        return it->second;
    }
    return nullptr;
}

bool Library::borrowBook(int memberID, const std::string &isbn) {
    Member* member = findMember(memberID);
    Book* book = findBook(isbn);

    if(!member) {
        std::cout << "Member not found!" << std::endl;
        return false;
    }
    if(!book) {
        std::cout << "Book not found!" << std::endl;
        return false;
    }
    if(!book->isAvailable()) {
        std::cout << "Book is already borrowed!" << std::endl;
        return false;
    }
    if(member->getBorrowedCount() >= 5) {
        std::cout << "Member has reached borrowing limit (5 books)!" <<
            std::endl;
        return false;
    }

    // All checks passed
    book->setAvailable(false);
    member->borrowBook(isbn);
    transactions.emplace_back(memberID, isbn, time(nullptr));
```

```

    std::cout << "Book borrowed successfully! Due date: 14 days from today."
              << std::endl;
    return true;
}

void Library::searchBooks(const std::string &keyword) const {
    bool found = false;
    std::cout << "\n=== Search Results ===" << std::endl;

    for(const auto &bookPtr : books) {
        if(bookPtr->matches(keyword)) {
            bookPtr->display();
            found = true;
        }
    }

    if(!found) {
        std::cout << "No books found matching '" << keyword << "'" << std::endl;
    }
}

void Library::generateStatistics() const {
    size_t total = books.size();
    size_t available = std::count_if(books.begin(), books.end(),
        [](const auto &b) { return b->isAvailable(); });

    std::cout << "\n=== Library Statistics ===" << std::endl;
    std::cout << "Total books: " << total << std::endl;
    std::cout << "Available: " << available << std::endl;
    std::cout << "Borrowed: " << (total - available) << std::endl;
    std::cout << "Total members: " << members.size() << std::endl;
    std::cout << "Active transactions: " << transactions.size() << std::endl;
}
```

3.2.4 Step 4: Complete Main Program

```
// main.cpp
#include "library.hpp"
#include <iostream>
#include <limits>

void clearInput() {
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

int main() {
    Library lib;

    // Pre-populate with some data
    lib.addBook("978-0-13-235088-4", "Clean Code", "Robert C. Martin", 2008);
```

```
    lib.addBook("978-0-13-468599-1", "Effective Modern C++", "Scott Meyers", 2014);
    lib.addBook("978-0-321-99278-9", "C++ Primer", "Stanley Lippman", 2012);

    lib.registerMember(1001, "Rahul Kumar", "rahul@email.com");
    lib.registerMember(1002, "Priya Singh", "priya@email.com");

    int choice;
    do {
        std::cout << "\n========== LIBRARY MANAGEMENT SYSTEM ==========" << std::endl;
        std::cout << "1.  Add New Book" << std::endl;
        std::cout << "2.  Remove Book" << std::endl;
        std::cout << "3.  Search Books" << std::endl;
        std::cout << "4.  List All Books" << std::endl;
        std::cout << "5.  Register Member" << std::endl;
        std::cout << "6.  Borrow Book" << std::endl;
        std::cout << "7.  Return Book" << std::endl;
        std::cout << "8.  View Statistics" << std::endl;
        std::cout << "0.  Exit" << std::endl;
        std::cout << "================================================" << std::endl;
        std::cout << "Enter choice: ";

        if(!(std::cin >> choice)) {
            clearInput();
            std::cout << "Invalid input!" << std::endl;
            continue;
        }
        clearInput();  // Clear remaining newline

        switch(choice) {
            case 1: {
                std::string isbn, title, author;
                int year;
                std::cout << "Enter ISBN: ";
                std::getline(std::cin, isbn);
                std::cout << "Enter title: ";
                std::getline(std::cin, title);
                std::cout << "Enter author: ";
                std::getline(std::cin, author);
                std::cout << "Enter year: ";
                std::cin >> year;
                lib.addBook(isbn, title, author, year);
                break;
            }
            case 2: {
                std::string isbn;
                std::cout << "Enter ISBN to remove: ";
                std::getline(std::cin, isbn);
                lib.removeBook(isbn);
                break;
            }
            case 3: {
                std::string keyword;
                std::cout << "Enter search keyword: ";
                std::getline(std::cin, keyword);
```

```
                lib.searchBooks(keyword);
                break;
            }
            case 4:
                lib.listAllBooks();
                break;
            case 5: {
                int id;
                std::string name, email;
                std::cout << "Enter member ID: ";
                std::cin >> id;
                clearInput();
                std::cout << "Enter name: ";
                std::getline(std::cin, name);
                std::cout << "Enter email: ";
                std::getline(std::cin, email);
                lib.registerMember(id, name, email);
                break;
            }
            case 6: {
                int memberID;
                std::string isbn;
                std::cout << "Enter member ID: ";
                std::cin >> memberID;
                clearInput();
                std::cout << "Enter ISBN: ";
                std::getline(std::cin, isbn);
                lib.borrowBook(memberID, isbn);
                break;
            }
            case 7: {
                int memberID;
                std::string isbn;
                std::cout << "Enter member ID: ";
                std::cin >> memberID;
                clearInput();
                std::cout << "Enter ISBN: ";
                std::getline(std::cin, isbn);
                lib.returnBook(memberID, isbn);
                break;
            }
            case 8:
                lib.generateStatistics();
                break;
            case 0:
                std::cout << "Thank you for using Library System!" <<
                    std::endl;
                break;
            default:
                std::cout << "Invalid choice!" << std::endl;
        }
    } while(choice != 0);

    return 0;
}
```

Pro Tip: Mindset Shift

What This Project Teaches:
OOP Concepts Used:

- Encapsulation (private data, public interface)
- Composition (Library has Books and Members)
- Association (Member borrows Books)
- Smart pointers (unique_ptr for ownership)

STL Used:

- vector for collections
- map for indexing (ISBN lookup)
- algorithm (find_if, count_if)
- string operations

Design Patterns:

- Manager pattern (Library class)
- Index pattern (isbnIndex for O(1) lookup)

Total: 400 lines, but professional structure!

Chapter 4

Project 2: Banking System - Inheritance and Polymorphism

4.1 Learning Focus

How to Think About This Problem

Goal: Master inheritance, polymorphism, and abstract classes.
The Scenario:

- Different account types: Savings, Current, Fixed Deposit
- All have: deposit, withdraw, check balance
- But different rules: Savings has interest, Current has overdraft, FD has lock-in period

Design:

- Abstract base: `Account` (pure virtual functions)
- Derived: `SavingsAccount`, `CurrentAccount`, `FixedDeposit`
- Bank manager works with `Account*` pointers (polymorphism)

4.2 Step-by-Step Build

4.2.1 Step 1: Abstract Account Class

```cpp
// account.hpp
#ifndef ACCOUNT_HPP
#define ACCOUNT_HPP

#include <string>
#include <iostream>

class Account {
protected:
    int accountNumber;
```

```
    std::string holderName;
    double balance;
    static int nextAccountNumber;  // For auto-generating

public:
    Account(std::string name, double initialDeposit);
    virtual ~Account() = default;  // Virtual destructor important!

    // Common operations (non-virtual)
    void deposit(double amount);
    bool withdraw(double amount);  // Basic version, virtual later

    // Pure virtual - must implement in derived
    virtual void calculateInterest() = 0;
    virtual void displayDetails() const = 0;
    virtual std::string getAccountType() const = 0;

    // Virtual with default implementation
    virtual bool canWithdraw(double amount) const;

    // Getters
    int getAccountNumber() const { return accountNumber; }
    double getBalance() const { return balance; }
    std::string getHolderName() const { return holderName; }
};

#endif
```

```
// account.cpp
#include "account.hpp"

int Account::nextAccountNumber = 1000;

Account::Account(std::string name, double initialDeposit)
    : holderName(name), balance(initialDeposit) {
    accountNumber = nextAccountNumber++;
}

void Account::deposit(double amount) {
    if(amount > 0) {
        balance += amount;
        std::cout << "Deposited: " << amount << std::endl;
    }
}

bool Account::withdraw(double amount) {
    if(canWithdraw(amount)) {
        balance -= amount;
        std::cout << "Withdrawn: " << amount << std::endl;
        return true;
    }
    std::cout << "Withdrawal failed!" << std::endl;
    return false;
}

bool Account::canWithdraw(double amount) const {
    return amount > 0 && amount <= balance;
}
```

4.2.2 Step 2: Derived Account Types

```cpp
// savings_account.hpp
#ifndef SAVINGS_ACCOUNT_HPP
#define SAVINGS_ACCOUNT_HPP

#include "account.hpp"

class SavingsAccount : public Account {
private:
    double interestRate;  // 4% per year
    int withdrawalLimit;  // 3 per month

public:
    SavingsAccount(std::string name, double initialDeposit);

    void calculateInterest() override;
    void displayDetails() const override;
    std::string getAccountType() const override { return "Savings"; }

    bool canWithdraw(double amount) const override;  // Check limit
};

#endif
```

```cpp
// savings_account.cpp
#include "savings_account.hpp"

SavingsAccount::SavingsAccount(std::string name, double initialDeposit)
    : Account(name, initialDeposit), interestRate(0.04), withdrawalLimit
        (3) {}

void SavingsAccount::calculateInterest() {
    double interest = balance * interestRate;
    balance += interest;
    std::cout << "Interest added: " << interest << std::endl;
}

void SavingsAccount::displayDetails() const {
    std::cout << "=== Savings Account ===" << std::endl;
    std::cout << "Account #: " << accountNumber << std::endl;
    std::cout << "Holder: " << holderName << std::endl;
    std::cout << "Balance: " << balance << std::endl;
    std::cout << "Interest Rate: " << (interestRate * 100) << "%" << std
        ::endl;
}

bool SavingsAccount::canWithdraw(double amount) const {
    if(withdrawalLimit <= 0) {
        std::cout << "Withdrawal limit reached for this period!" << std
            ::endl;
        return false;
    }
    return Account::canWithdraw(amount);  // Call base check
}
```

```cpp
// current_account.hpp
#ifndef CURRENT_ACCOUNT_HPP
```

```cpp
#define CURRENT_ACCOUNT_HPP

#include "account.hpp"

class CurrentAccount : public Account {
private:
    double overdraftLimit;  // Can go negative up to this

public:
    CurrentAccount(std::string name, double initialDeposit);

    void calculateInterest() override;  // No interest
    void displayDetails() const override;
    std::string getAccountType() const override { return "Current"; }

    bool canWithdraw(double amount) const override;
};

#endif
```

```cpp
// current_account.cpp
#include "current_account.hpp"

CurrentAccount::CurrentAccount(std::string name, double initialDeposit)
    : Account(name, initialDeposit), overdraftLimit(10000.0) {}

void CurrentAccount::calculateInterest() {
    std::cout << "No interest for current accounts." << std::endl;
}

void CurrentAccount::displayDetails() const {
    std::cout << "=== Current Account ===" << std::endl;
    std::cout << "Account #: " << accountNumber << std::endl;
    std::cout << "Holder: " << holderName << std::endl;
    std::cout << "Balance: " << balance << std::endl;
    std::cout << "Overdraft Limit: " << overdraftLimit << std::endl;
}

bool CurrentAccount::canWithdraw(double amount) const {
    // Can withdraw if balance + overdraft >= amount
    return amount > 0 && (balance + overdraftLimit) >= amount;
}
```

4.2.3 Step 3: Bank Class (Polymorphic Container)

```cpp
// bank.hpp
#ifndef BANK_HPP
#define BANK_HPP

#include "account.hpp"
#include <vector>
#include <memory>

class Bank {
private:
    std::string bankName;
```

```
    std::vector<std::unique_ptr<Account>> accounts;

public:
    explicit Bank(std::string name);

    // Factory methods - create and store
    void openSavingsAccount(std::string holder, double deposit);
    void openCurrentAccount(std::string holder, double deposit);

    // Polymorphic operations
    Account* findAccount(int accountNumber);
    void displayAllAccounts() const;
    void applyInterestToAll();
    void processMonthlyMaintenance();

    // Statistics
    double getTotalDeposits() const;
    void generateReport() const;
};

#endif
```

```
// bank.cpp
#include "bank.hpp"
#include "savings_account.hpp"
#include "current_account.hpp"
#include <numeric>

Bank::Bank(std::string name) : bankName(name) {}

void Bank::openSavingsAccount(std::string holder, double deposit) {
    accounts.push_back(std::make_unique<SavingsAccount>(holder, deposit));
    std::cout << "Savings account opened. Number: "
              << accounts.back()->getAccountNumber() << std::endl;
}

void Bank::openCurrentAccount(std::string holder, double deposit) {
    accounts.push_back(std::make_unique<CurrentAccount>(holder, deposit));
    std::cout << "Current account opened. Number: "
              << accounts.back()->getAccountNumber() << std::endl;
}

Account* Bank::findAccount(int accountNumber) {
    for(auto &acc : accounts) {
        if(acc->getAccountNumber() == accountNumber) {
            return acc.get();
        }
    }
    return nullptr;
}

void Bank::displayAllAccounts() const {
    std::cout << "\n=== " << bankName << " - All Accounts ===" << std::endl;
    for(const auto &acc : accounts) {
        acc->displayDetails();  // Polymorphic call!
```

```
        std::cout << std::endl;
    }
}

void Bank::applyInterestToAll() {
    for(auto &acc : accounts) {
        acc->calculateInterest();  // Different behavior for each type!
    }
}

double Bank::getTotalDeposits() const {
    double total = 0;
    for(const auto &acc : accounts) {
        total += acc->getBalance();
    }
    return total;
}

void Bank::generateReport() const {
    size_t savingsCount = 0, currentCount = 0;

    for(const auto &acc : accounts) {
        if(acc->getAccountType() == "Savings") savingsCount++;
        else if(acc->getAccountType() == "Current") currentCount++;
    }

    std::cout << "\n========␣BANK␣REPORT␣========" << std::endl;
    std::cout << "Bank:␣" << bankName << std::endl;
    std::cout << "Total␣Accounts:␣" << accounts.size() << std::endl;
    std::cout << "␣␣-␣Savings:␣" << savingsCount << std::endl;
    std::cout << "␣␣-␣Current:␣" << currentCount << std::endl;
    std::cout << "Total␣Deposits:␣" << getTotalDeposits() << std::endl;
    std::cout << "================================" << std::endl;
}
```

4.2.4 Step 4: Main Program Showing Polymorphism

```
// bank_main.cpp
#include "bank.hpp"
#include <iostream>

int main() {
    Bank myBank("National␣C++␣Bank");

    // Open different account types
    myBank.openSavingsAccount("Rahul␣Kumar", 50000);
    myBank.openCurrentAccount("Priya␣Enterprises", 100000);
    myBank.openSavingsAccount("Amit␣Singh", 25000);

    // Display all - notice different outputs!
    myBank.displayAllAccounts();

    // Apply interest - different calculations!
    std::cout << "\n>>>␣Applying␣Annual␣Interest␣<<<" << std::endl;
    myBank.applyInterestToAll();

```

```
    // Find and operate on specific account
    int accNum;
    std::cout << "\nEnter account number for withdrawal: ";
    std::cin >> accNum;

    Account* acc = myBank.findAccount(accNum);
    if(acc) {
        acc->displayDetails();

        double amount;
        std::cout << "Enter amount to withdraw: ";
        std::cin >> amount;

        // Polymorphic behavior: Savings checks limit, Current allows
            overdraft
        acc->withdraw(amount);

        acc->displayDetails();
    } else {
        std::cout << "Account not found!" << std::endl;
    }

    // Final report
    myBank.generateReport();

    return 0;
}
```

Pro Tip: Mindset Shift

Key Polymorphism Insights:

1. **Base pointer, derived behavior:**
   - `Account* acc = new SavingsAccount(...)`
   - `acc->calculateInterest()` calls Savings version
2. **Container of base pointers holds all types:**
   - `vector<unique_ptr<Account>>` holds both Savings and Current
3. **Virtual destructor essential:**
   - Without it, deleting derived through base pointer leaks memory

Design benefit: Add `FixedDepositAccount` tomorrow without changing Bank class!

Chapter 5

Project 3: E-Commerce Console Application

5.1 The Ultimate Integration Project

How to Think About This Problem

Concepts to integrate:

- All OOP concepts (encapsulation, inheritance, polymorphism)
- STL containers (vector, map, set, queue, stack)
- Algorithms (sorting, searching, filtering)
- File I/O (persistence)
- Exception handling
- Template basics (generic container wrapper)

System Design:

- **Product hierarchy:** Electronics, Clothing, Books (inheritance)
- **User system:** Customer, Admin (inheritance + permissions)
- **Cart:** Stack-like (LIFO for undo), queue for checkout processing
- **Order management:** Map for O(1) lookup by order ID
- **Inventory:** Set for unique categories, priority queue for restocking

5.2 Core Classes Overview

```
// product.hpp - Product hierarchy
#ifndef PRODUCT_HPP
#define PRODUCT_HPP

```

```cpp
#include <string>
#include <iostream>

class Product {
protected:
    int productID;
    std::string name;
    std::string description;
    double price;
    int stockQuantity;
    std::string category;

public:
    Product(int id, std::string n, std::string desc, double p, int stock, std::string cat);
    virtual ~Product() = default;

    // Getters
    int getID() const;
    std::string getName() const;
    double getPrice() const;
    int getStock() const;
    std::string getCategory() const;

    // Stock management
    bool reduceStock(int qty);
    void addStock(int qty);

    // Virtual methods
    virtual void display() const;
    virtual double getDiscountedPrice() const;  // Default: no discount
    virtual std::string getType() const = 0;
};

class Electronics : public Product {
private:
    int warrantyMonths;
    std::string brand;

public:
    Electronics(int id, std::string n, std::string desc, double p, int stock,
                std::string brand, int warranty);

    void display() const override;
    double getDiscountedPrice() const override;  // 10% off electronics
    std::string getType() const override { return "Electronics"; }
};

class Clothing : public Product {
private:
    std::string size;
    std::string color;
    std::string material;

public:
    Clothing(int id, std::string n, std::string desc, double p, int stock,
```

```
              std::string size, std::string color, std::string material);

    void display() const override;
    double getDiscountedPrice() const override;  // Seasonal discount
    std::string getType() const override { return "Clothing"; }

    bool isSizeAvailable(const std::string &s) const;
};

class Book : public Product {
private:
    std::string author;
    std::string isbn;
    int publicationYear;

public:
    Book(int id, std::string n, std::string desc, double p, int stock,
         std::string author, std::string isbn, int year);

    void display() const override;
    double getDiscountedPrice() const override;  // Student discount
    std::string getType() const override { return "Book"; }
};

#endif
```

```
// cart.hpp - Shopping cart with undo feature
#ifndef CART_HPP
#define CART_HPP

#include "product.hpp"
#include <vector>
#include <stack>
#include <map>

struct CartItem {
    const Product* product;  // Pointer to product in catalog
    int quantity;
    double priceAtAddition;  // Snapshot price (for price protection)
};

class ShoppingCart {
private:
    std::vector<CartItem> items;
    std::stack<std::vector<CartItem>> undoStack;  // For undo feature
    double total;

public:
    ShoppingCart();

    void addItem(const Product* prod, int qty);
    void removeItem(int productID);
    void updateQuantity(int productID, int newQty);
    void undo();  // Restore previous state

    void display() const;
    double getTotal() const;
    bool isEmpty() const;
```

```
    void clear();

    std::vector<CartItem> getItems() const { return items; }
};

#endif
```

```
// order.hpp - Order processing system
#ifndef ORDER_HPP
#define ORDER_HPP

#include "cart.hpp"
#include <ctime>
#include <map>

enum class OrderStatus {
    PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED
};

class Order {
private:
    int orderID;
    int customerID;
    std::vector<CartItem> items;
    double totalAmount;
    double tax;
    double finalAmount;
    OrderStatus status;
    time_t orderDate;
    std::string shippingAddress;

public:
    Order(int oid, int cid, const ShoppingCart &cart, std::string
        address);

    void updateStatus(OrderStatus newStatus);
    void display() const;

    // Getters
    int getID() const { return orderID; }
    int getCustomerID() const { return customerID; }
    OrderStatus getStatus() const { return status; }
    double getTotal() const { return finalAmount; }
};

class OrderManager {
private:
    std::map<int, Order> orders;  // orderID -> Order
    int nextOrderID;

public:
    OrderManager() : nextOrderID(1000) {}

    int createOrder(int customerID, const ShoppingCart &cart, std::
        string address);
    Order* findOrder(int orderID);
    void updateOrderStatus(int orderID, OrderStatus status);
    void displayCustomerOrders(int customerID) const;
```

```
    void displayAllOrders() const;

    // Analytics
    double getTotalRevenue() const;
    std::map<OrderStatus, int> getOrderStatistics() const;
};

#endif
```

5.3 The Complete System

```
// ecommerce_system.hpp
#ifndef ECOMMERCE_SYSTEM_HPP
#define ECOMMERCE_SYSTEM_HPP

#include "product.hpp"
#include "cart.hpp"
#include "order.hpp"
#include <vector>
#include <map>
#include <memory>
#include <set>

class ECommerceSystem {
private:
    // Product catalog - polymorphic storage
    std::vector<std::unique_ptr<Product>> catalog;
    std::map<int, Product*> productIndex;  // Fast lookup

    // User management
    std::map<int, std::string> customers;  // ID -> Name
    int nextCustomerID;

    // Active carts
    std::map<int, ShoppingCart> activeCarts;  // CustomerID -> Cart

    // Orders
    OrderManager orderManager;

    // Categories for browsing
    std::set<std::string> categories;

public:
    ECommerceSystem();

    // Catalog management
    void addElectronics(std::string name, std::string desc, double price,
                        int stock, std::string brand, int warranty);
    void addClothing(std::string name, std::string desc, double price,
                     int stock, std::string size, std::string color, std::string material);
    void addBook(std::string name, std::string desc, double price,
                 int stock, std::string author, std::string isbn, int year);

```

```
    void displayCatalog() const;
    void displayByCategory(const std::string &cat) const;
    void searchProducts(const std::string &keyword) const;
    Product* findProduct(int id);

    // Customer operations
    int registerCustomer(std::string name);
    ShoppingCart& getCart(int customerID);

    // Checkout
    int checkout(int customerID, std::string address);

    // Order tracking
    void viewOrder(int orderID);
    void viewCustomerHistory(int customerID);

    // Admin
    void generateInventoryReport() const;
    void generateSalesReport() const;

    // Data persistence (simplified)
    void saveToFile(const std::string &filename);
    void loadFromFile(const std::string &filename);
};

#endif
```

5.4 Sample Usage (Main Program)

```
// ecommerce_main.cpp
#include "ecommerce_system.hpp"
#include <iostream>

void showMenu() {
    std::cout << "\n========== E-COMMERCE SYSTEM ==========" << std::
        endl;
    std::cout << "1.  Browse All Products" << std::endl;
    std::cout << "2.  Browse by Category" << std::endl;
    std::cout << "3.  Search Products" << std::endl;
    std::cout << "4.  View Product Details" << std::endl;
    std::cout << "5.  Add to Cart" << std::endl;
    std::cout << "6.  View Cart" << std::endl;
    std::cout << "7.  Remove from Cart" << std::endl;
    std::cout << "8.  Undo Last Cart Action" << std::endl;
    std::cout << "9.  Checkout" << std::endl;
    std::cout << "10. View Order Status" << std::endl;
    std::cout << "11. Admin: Inventory Report" << std::endl;
    std::cout << "12. Admin: Sales Report" << std::endl;
    std::cout << "0.  Exit" << std::endl;
    std::cout << "=========================================" << std::endl;
}

int main() {
    ECommerceSystem shop;

    // Setup inventory
```

```
    shop.addElectronics("iPhone 15", "Latest Apple smartphone", 79999, 50,
                        "Apple", 12);
    shop.addElectronics("MacBook Pro", "Professional laptop", 199999, 30,
                        "Apple", 24);
    shop.addClothing("Cotton T-Shirt", "Premium quality casual wear", 999, 100,
                     "M", "Blue", "100% Cotton");
    shop.addClothing("Formal Shirt", "Office wear", 1499, 75,
                     "L", "White", "Cotton Blend");
    shop.addBook("C++ Primer", "Comprehensive C++ guide", 899, 200,
                 "Stanley Lippman", "978-0321714114", 2012);
    shop.addBook("Clean Code", "Software engineering best practices", 699, 150,
                 "Robert Martin", "978-0132350884", 2008);

    // Register customer
    int customerID = shop.registerCustomer("Rahul Kumar");
    std::cout << "Welcome! Your customer ID is: " << customerID << std::endl;

    int choice;
    do {
        showMenu();
        std::cout << "Enter choice: ";
        std::cin >> choice;

        switch(choice) {
            case 1:
                shop.displayCatalog();
                break;
            case 2: {
                std::string cat;
                std::cout << "Enter category (Electronics/Clothing/Book): ";
                std::cin >> cat;
                shop.displayByCategory(cat);
                break;
            }
            case 3: {
                std::string keyword;
                std::cout << "Enter search term: ";
                std::cin >> keyword;
                shop.searchProducts(keyword);
                break;
            }
            case 4: {
                int pid;
                std::cout << "Enter product ID: ";
                std::cin >> pid;
                Product* p = shop.findProduct(pid);
                if(p) p->display();
                break;
            }
            case 5: {
                int pid, qty;
                std::cout << "Enter product ID and quantity: ";
```

```
                std::cin >> pid >> qty;
                Product* p = shop.findProduct(pid);
                if(p) {
                    shop.getCart(customerID).addItem(p, qty);
                    std::cout << "Added to cart!" << std::endl;
                }
                break;
            }
            case 6:
                shop.getCart(customerID).display();
                break;
            case 9: {
                std::string address;
                std::cout << "Enter shipping address: ";
                std::cin.ignore();
                std::getline(std::cin, address);
                int orderID = shop.checkout(customerID, address);
                if(orderID != -1) {
                    std::cout << "Order placed! ID: " << orderID << std
                        ::endl;
                }
                break;
            }
            case 10: {
                int oid;
                std::cout << "Enter order ID: ";
                std::cin >> oid;
                shop.viewOrder(oid);
                break;
            }
            case 11:
                shop.generateInventoryReport();
                break;
            case 12:
                shop.generateSalesReport();
                break;
        }
    } while(choice != 0);

    return 0;
}
```

Pro Tip: Mindset Shift

What You've Learned:
OOP Mastery:

- Inheritance hierarchy (Product → Electronics/Clothing/Book)
- Polymorphic containers (vector<unique_ptr<Product»)
- Abstract classes and virtual functions
- Proper use of const, references, smart pointers

STL Mastery:

- vector, map, set, stack, queue in real context
- Algorithms: find_if, sort, accumulate, count_if
- Iterators and range-based for

Software Engineering:

- Separation of concerns (Cart, Order, Product, System)
- Indexing for performance (O(1) lookups)
- State management (undo stack, order status)
- Data persistence concepts

Total System: 800 lines, professional architecture!

Chapter 6

How to Continue Learning

6.1 Self-Learning Strategies

When You Get Stuck: Debug Strategy

When You Don't Know How to Implement Something:

1. **Break it smaller:** "How to build e-commerce?" → "How to store products?" → "How to use vector?"
2. **Find similar examples:**
   - Search: "C++ vector of polymorphic objects"
   - cppreference.com → Examples section
   - GitHub: Search "student management system cpp" (read, don't copy)
3. **Prototype in isolation:**
   - New file: just test map<string, int>
   - Understand, then integrate
4. **rubber duck debugging:** Explain code line-by-line to imaginary friend
5. **When stuck for 30+ minutes:**
   - Take 10 min break
   - Write down exactly what's confusing
   - Search that specific error/uncertainty

Pro Tip: Mindset Shift

Next Steps After This Guide:

1. **Build your own version:** Don't look at our code, rebuild from memory
2. **Add features:**
   - File persistence (save/load to text file)

- Template-based generic containers
- Exception handling for all errors
- Unit tests using simple assert statements

3. **GUI Version:** Learn Qt or ncurses for graphical interface
4. **Network Version:** Convert to client-server (socket programming)
5. **Database Version:** Connect to SQLite instead of memory storage

Remember: Every professional started with "Hello World". The difference is they built 100 small programs before attempting big ones.

Code. Fail. Learn. Repeat.

"The only way to learn a new programming language is by writing programs in it." - Dennis Ritchie

---